


Universidad de Concepción

Facultad de Ingeniería
Departamento de Ingeniería Informática y Ciencias de la Computación


Estructuras de Datos y Algoritmos Avanzados (2026-2)

# Boletín 03: String Matching


**Autor:**
Bryan Eliseo Aguirre Fuentes

**Matrícula:**
2024443402

**Profesor:**
Dr. José Fuentes

**Ayudante:**
Leonardo Lovera



Concepción, Chile
6 de noviembre de 2026

# Índice

**1. Resumen** **1**
1.1. Propósito y alcance del estudio . . . 1
1.2. Estructuras y algoritmos estudiados . . . 1

**2. Análisis teórico** **2**
2.1. Búsqueda por fuerza bruta . . . 2
2.2. Knuth–Morris–Pratt (KMP) . . . 2
2.3. Rabin–Karp . . . 2
2.4. Suffix Array . . . 3
2.5. Comparación de complejidades . . . 3

**3. Implementación** **4**
3.1. Búsqueda por fuerza bruta . . . 4
3.2. Knuth–Morris–Pratt (KMP) . . . 4
3.3. Rabin–Karp . . . 4
3.4. Suffix Array . . . 4

**4. Metodología experimental** **5**
4.1. Características de la máquina . . . 5
4.2. Datos de entrada . . . 5
4.3. Experimentos e hipótesis . . . 5
4.3.1. Efecto del largo del texto . . . 6
4.3.2. Efecto del largo del patrón . . . 6
4.3.3. Efecto del número de ocurrencias . . . 7

**Referencias** **8**

# 1. Resumen

## 1.1. Propósito y alcance del estudio

Este estudio compara experimentalmente cuatro métodos de búsqueda de patrones en texto: búsqueda por fuerza bruta, Suffix Array, Knuth–Morris–Pratt (KMP) y Rabin–Karp. La fuerza bruta se incluye como **línea base** para contextualizar el desempeño de los métodos estudiados. Para efectos de los experimentos, *búsqueda* significa encontrar y contar todas las ocurrencias del patrón en el texto, no devolver sus posiciones. Se analiza cómo influyen el largo del texto, el largo del patrón y el número de ocurrencias en el tiempo de búsqueda; además, se consideran el costo de construcción del índice y el consumo de memoria. Los resultados se interpretan junto con las complejidades teóricas para identificar los escenarios en que conviene cada alternativa.

## 1.2. Estructuras y algoritmos estudiados

**Fuerza bruta:** Revisa cada posición posible del texto y compara desde allí los caracteres con el patrón. No requiere preprocesamiento y sirve como referencia sencilla, aunque puede repetir comparaciones.

**Suffix Array:** Ordena las posiciones iniciales de los sufijos del texto. Para buscar un patrón, localiza mediante búsqueda binaria el intervalo de sufijos que comienza con él y cuenta sus ocurrencias. Su índice requiere una etapa de construcción previa.

**KMP:** Busca el patrón recorriendo el texto y usando información de prefijos del patrón para evitar comparaciones repetidas. Su búsqueda tiene tiempo lineal en el largo del texto y del patrón, sin requerir un índice persistente.

**Rabin–Karp:** Compara primero valores hash del patrón y de ventanas del texto; cuando coinciden, verifica los caracteres para confirmar la ocurrencia. El hash de las ventanas se actualiza incrementalmente y su desempeño depende, entre otros factores, de las colisiones.

# 2. Análisis teórico

Sea $T$ el texto y $P$ el patrón. Denotamos sus longitudes por $n = |T|$ y $m = |P|$, respectivamente. Excluimos el patrón vacío ($m = 0$), cuyo número de coincidencias depende de la convención adoptada. Analizamos el caso usual $1 \leq m \leq n$; si $m > n$, las implementaciones devuelven cero inmediatamente. Por *búsqueda* entendemos contar todas las ocurrencias, incluidas las superpuestas. Usamos notación $O(\cdot)$ para expresar cotas superiores y $\Theta(\cdot)$ cuando la cota es ajustada. El espacio se expresa en palabras y no incluye la entrada. Como los métodos devuelven un contador, el espacio de salida es $O(1)$; si devolvieran todas las posiciones, habría que sumar $O(z)$, donde $z$ es el número de ocurrencias.

## 2.1. Búsqueda por fuerza bruta

El algoritmo prueba cada posición de $T$ y compara desde allí los caracteres de $P$, avanzando una posición tras cada intento.

- **Preparación:** $\Theta(1)$; no construye estructuras previas.
- **Mejor caso de búsqueda:** $\Theta(n - m + 1)$, cuando cada intento falla al comparar el primer carácter. Una coincidencia inicial puede añadir hasta $O(m)$ comparaciones, pero el algoritmo debe revisar las posiciones restantes para contar todas las ocurrencias.
- **Peor caso de búsqueda:** $\Theta((n - m + 1)m)$, si cada ventana coincide en casi todo el patrón antes de fallar o confirmarse.
- **Memoria auxiliar:** $\Theta(1)$.

## 2.2. Knuth–Morris–Pratt (KMP)

KMP construye una tabla que indica qué prefijo del patrón puede conservarse después de una discrepancia. Así desplaza el patrón sin retroceder en el texto. La implementación construye la tabla en cada llamada y luego recorre el texto; tras una coincidencia, también usa la tabla para conservar posibles coincidencias superpuestas.

- **Preparación e inicialización:** $\Theta(m)$ tiempo y memoria para construir la tabla de prefijos.
- **Mejor caso de búsqueda:** $\Theta(n)$.
- **Peor caso de búsqueda:** $\Theta(n)$; el recorrido avanza por el texto y los retrocesos del patrón son amortizadamente lineales. Incluida la preparación, el tiempo es $\Theta(n + m)$.
- **Memoria auxiliar:** $\Theta(m)$, por la tabla.

## 2.3. Rabin–Karp

Rabin–Karp calcula el hash del patrón $P$ y el de la primera ventana de $T$ durante la preparación. La búsqueda verifica esa primera ventana y luego desplaza la ventana, actualizando su hash en tiempo constante. Compara caracteres para confirmar cada coincidencia de hash.

- **Preparación e inicialización:** $\Theta(m)$ tiempo para calcular ambos hashes: el de $P$ y el de la primera ventana de $T$.

- **Mejor caso de búsqueda:** $\Theta(n - m + 1)$, cuando el hash de $P$ difiere del hash de todas las ventanas examinadas. No hay verificaciones carácter a carácter y cada ventana se procesa en tiempo constante.
- **Peor caso de búsqueda:** $\Theta((n - m + 1)m)$, si muchas ventanas requieren una verificación completa, por colisiones o coincidencias reales superpuestas.
- **Memoria auxiliar:** $\Theta(1)$.

El peor caso puede ocurrir por colisiones de hash o por coincidencias reales en muchas ventanas. Como se usan parámetros de hash fijos y no se supone una distribución aleatoria de las entradas, no se afirma una garantía de tiempo esperado. Se asume que las operaciones con el hash modular cuestan tiempo constante.

### 2.4. Suffix Array

La estructura ordena los sufijos de $T$ mediante duplicación de prefijos: en cada ronda duplica el largo del prefijo ordenado y actualiza clases con conteo. Para contar un patrón, realiza dos búsquedas binarias que delimitan los sufijos que comienzan con $P$; no recorre individualmente las $z$ ocurrencias.

- **Preparación (construcción):** $O(n \log n)$ tiempo en el peor caso y $\Theta(n)$ memoria, mediante duplicación de prefijos y ordenamiento por conteo en cada ronda. Esta implementación ejecuta todas las rondas, por lo que su construcción toma $\Theta(n \log n)$.
- **Mejor caso de búsqueda:** $\Theta(\log n)$, si las comparaciones binarias terminan al revisar el primer carácter.
- **Peor caso de búsqueda:** $O(m \log n)$, pues cada comparación puede revisar hasta $m$ caracteres. Es la búsqueda binaria básica, sin optimizaciones basadas en LCP; contar el intervalo cuesta $O(1)$.
- **Memoria:** $O(1)$ auxiliar por consulta; el índice persistente ocupa $\Theta(n)$ y la memoria temporal de construcción, $\Theta(n)$.

La memoria depende de los largos de entrada, no de su contenido; por ello no cambia entre mejor y peor caso.

### 2.5. Comparación de complejidades

Los tiempos de búsqueda incluyen el conteo. En Rabin–Karp, la preparación incluye el hash de $P$ y el de la primera ventana de $T$. La preparación del Suffix Array corresponde a su construcción; se distingue el espacio auxiliar por consulta del índice persistente y de la memoria temporal de construcción.

| **Método** | **Preparación** | **Mejor búsqueda** | **Peor búsqueda** | **Memoria auxiliar / índice** |
|---|---|---|---|---|
| Fuerza bruta | $\Theta(1)$ | $\Theta(n - m + 1)$ | $\Theta((n - m + 1)m)$ | $\Theta(1)$ |
| KMP | $\Theta(m)$ | $\Theta(n)$ | $\Theta(n)$ | $\Theta(m)$ |
| Rabin–Karp | $\Theta(m)$ | $\Theta(n - m + 1)$ | $\Theta((n - m + 1)m)$ | $\Theta(1)$ |
| Suffix Array | $\Theta(n \log n)$ | $\Theta(\log n)$ | $O(m \log n)$ | $O(1)$ consulta; $\Theta(n)$ índice persistente; $\Theta(n)$ temporal al construir |

# 3. Implementación

Los algoritmos directos reciben primero el patrón y luego el texto, y devuelven un contador de ocurrencias; Suffix Array se construye con el texto y se consulta con el patrón. Fuerza bruta se incorporó como línea base; KMP, Rabin–Karp y Suffix Array se obtuvieron del repositorio del curso. Las cuatro implementaciones cuentan coincidencias superpuestas. Para conservar los bytes originales del corpus se corrigió la interpretación de caracteres como `unsigned char` en los índices del Suffix Array y en el hash de Rabin–Karp, sin sustituir sus algoritmos.

## 3.1. Búsqueda por fuerza bruta

Esta implementación se agregó para el estudio. La función `brute_force` recibe vistas `string_view` del patrón y del texto. Un ciclo `for` prueba cada posición inicial válida, y un ciclo `while` compara los caracteres correspondientes. Si el patrón coincide completo, incrementa `matches`; al avanzar una posición en cada iteración, cuenta también coincidencias superpuestas.

## 3.2. Knuth–Morris–Pratt (KMP)

La función `kmp` construye el vector `p` de largo $m + 1$, que guarda la información de prefijos y fallos del patrón. Un ciclo lo prepara; después, el ciclo de búsqueda usa `i` como índice del texto y `j` como índice del patrón. Ante una discrepancia, actualiza `j` con `p[j]` sin retroceder `i`; al completar una coincidencia, incrementa `matches` y reutiliza `p` para permitir solapamientos. La implementación proviene del repositorio del curso [2].

## 3.3. Rabin–Karp

La función `rk` usa `p` y `t` para los hashes del patrón y de la ventana actual, `q` como módulo y `d = 256` como base. Un ciclo inicial calcula ambos hashes; el ciclo principal recorre las ventanas. Cuando los hashes coinciden, un ciclo `while` verifica los caracteres y actualiza `count` si hay coincidencia. Después actualiza `t` en tiempo constante para la siguiente ventana. La implementación proviene del repositorio del curso [3].

## 3.4. Suffix Array

El constructor añade el centinela ETX al texto y crea el vector `SA` con las posiciones iniciales de los sufijos, además de los vectores `p`, `q` y `r` para posiciones temporales y clases. En cada ronda, un ciclo duplica el prefijo considerado; `count` acumula frecuencias para ordenar las clases. El método `count` busca con dos ciclos `while` los límites del intervalo de sufijos que comienza con el patrón, comparando con `substr` y `starts_with`; resta los límites para obtener el número de ocurrencias. La implementación proviene del repositorio del curso [1].

# 4. Metodología experimental

## 4.1. Características de la máquina

Los experimentos se realizarán en una estación de trabajo con las siguientes características:

- **Procesador:** Intel Core i5-13420H (Raptor Lake), 8 núcleos y 12 hilos (4 P-Cores y 4 E-Cores), frecuencia base de 2.10 GHz y turbo de hasta 4.60 GHz. Cachés: L1d de 320 KiB, L1i de 384 KiB, L2 de 7 MiB y L3 compartida de 12 MiB.
- **Memoria principal:** 16 GiB DDR (15 GiB utilizables por el sistema operativo).
- **Almacenamiento:** SSD NVMe WD PC SN740 de 512 GB, interfaz PCIe Gen4 $\times 4$.
- **Entorno de compilación:** Arch Linux x86_64 (kernel 6.x), GCC y las banderas `-std=c++20 -O3 -march=native`.
- **Afinidad de CPU:** se fijará el proceso de medición al núcleo lógico 2 mediante `taskset -c 2`. Esto reduce las migraciones entre núcleos, pero no elimina variaciones de frecuencia, temperatura ni la actividad concurrente del sistema.

## 4.2. Datos de entrada

Los textos procederán del archivo `english.50MB` de Pizza & Chili, formado por textos en inglés de Project Gutenberg [4]. Se usarán sus primeros 50 millones de bytes y prefijos de 1, 5, 10 y 20 millones de bytes; en este estudio, $1\,\text{MB} = 10^6$ bytes. Las longitudes $n$ y $m$ se medirán en bytes, conservando mayúsculas, espacios y puntuación.

Los patrones serán subcadenas del corpus, no necesariamente palabras completas. Antes de medir se verificarán sus frecuencias, incluidas las coincidencias superpuestas, y se conservarán sus bytes, longitudes y posiciones de extracción. Por ejemplo, se verificaron como únicas las subcadenas `his etext was produced by Normal` (32 bytes, posición 6) y `Normal W` (8 bytes, posición 32), contando las posiciones desde cero. Se conserva el archivo comprimido original y un manifiesto SHA-256; la preparación rechaza cambios de contenido. El prefijo verificado no contiene bytes de valor 0 a 3, por lo que se mantiene el centinela ETX del Suffix Array. Los bytes altos se interpretan sin signo, sin modificar el texto.

Como excepción obligatoria se generará $T = A^n$. Por ejemplo, `AAAAAAAAAA` contiene diez ocurrencias de $P =$ `A` y ninguna de $P =$ `B`.

## 4.3. Experimentos e hipótesis

Se compararán los cuatro métodos sobre las mismas entradas. La búsqueda de KMP y Rabin–Karp incluirá la preparación que sus funciones realizan en cada llamada; la consulta del Suffix Array usará el índice ya construido. Su construcción se medirá por separado, de modo que una consulta rápida no oculte ese costo inicial. La lectura del archivo, la selección de patrones y la validación de conteos quedarán fuera del tiempo medido.

El esquema de medición se basará en la plantilla `uhr`, de Leonardo Lovera (LELE), usando `std::chrono::steady_clock` y nanosegundos [5]. Cada patrón se medirá en un lote calibrado fuera de muestra para alcanzar al menos 5 ms o un máximo de un millón de consultas. El tamaño del lote se conservará entre repeticiones y su tiempo se dividirá por la cantidad de consultas. La construcción se expresará en tiempo por construcción, sin incluir la destrucción del índice.

Cada configuración tendrá **64 repeticiones**, después del calentamiento. Se usarán los mismos lotes de patrones para todos los métodos; los conteos se consumirán para evitar su eliminación por el compilador y se validarán fuera del intervalo medido. Se aplicará afinidad al núcleo lógico 2 y una pausa de dos segundos entre series. Se conservarán las muestras crudas y se generarán CSV con **media, desviación estándar muestral, mínimo, Q1, mediana, Q3 y máximo**, calculados sobre los tiempos normalizados de las repeticiones. Las figuras mostrarán media y barras de una desviación estándar, no intervalos de confianza; el extremo inferior se limitará a cero. En el experimento de ocurrencias se conservará el resumen por patrón para no mezclar conteos distintos. Las repeticiones temporales no se tratarán como patrones independientes. En Rabin–Karp se mantendrá $q = 101$ en todos los experimentos.

La memoria se medirá en una pasada independiente mediante seguimiento de los bytes solicitados al asignador por cadenas y vectores. Se distinguirán el pico auxiliar por consulta, el índice persistente, el pico total de construcción y el pico temporal de las reservas liberadas durante la construcción. Se excluyen la entrada compartida, los metadatos del asignador y los datos del medidor; el seguimiento permanece desactivado al cronometrar. Fuerza bruta y Rabin–Karp no reservan memoria dinámica auxiliar, aunque usan variables escalares constantes. Estas medidas no representan la memoria residente total del proceso.

Para graficar frente a $n$ y $m$, se promediarán con igual peso los tiempos de los patrones en cada repetición y se resumirán esos 64 promedios. Frente a $z$, cada patrón conservará su propio punto y dispersión. Se preservarán CSV crudos y resúmenes con los parámetros y hashes de cada sesión.

#### 4.3.1. Efecto del largo del texto

- **Variable:** $n$, con textos de 1, 5, 10, 20 y 50 MB.
- **Controles:** diez patrones distintos de $m = 32$ bytes, extraídos del primer MB y verificados como únicos en los 50 MB. Se usará exactamente el mismo conjunto en todos los tamaños, manteniendo $z = 1$. La selección se realizará antes de medir, sin insertar coincidencias.
- **Resultados:** tiempo por consulta frente a $n$, construcción del Suffix Array frente a $n$ y consumo de memoria. Se mostrarán gráficos y tablas; los conteos deberán coincidir entre métodos.

**Hipótesis.** Con $|P|$ y el número de ocurrencias fijos, se espera que fuerza bruta y KMP aumenten aproximadamente de forma lineal con el largo del texto. Rabin–Karp también debería crecer cerca de lineal si las colisiones son escasas. La consulta del Suffix Array debería crecer más lentamente, de forma aproximadamente logarítmica, aunque la construcción del índice crecerá como $n \log n$. El índice y su memoria temporal crecerán linealmente con $n$; la memoria auxiliar de los otros métodos permanecerá aproximadamente constante porque $m$ no cambia.

#### 4.3.2. Efecto del largo del patrón

- **Variable:** $m$, con longitudes de 8, 16, 32, 64, 128 y 256 bytes.
- **Controles:** un texto fijo de 50 MB y diez patrones distintos por longitud, cada uno con exactamente una ocurrencia verificada. Así se evita confundir patrones más largos con una disminución del conteo. El Suffix Array será el mismo para todas las longitudes.
- **Resultados:** tiempo por consulta y memoria frente a $m$. Se informará el costo de construcción del índice como costo fijo, sin incluirlo nuevamente en cada consulta.

**Hipótesis.** KMP conservará un costo dominado por recorrer el texto, con preparación y memoria lineales en $m$. Rabin–Karp sumará una preparación lineal en $m$, pero no necesariamente una búsqueda más lenta si hay pocas verificaciones. En fuerza bruta, el crecimiento de la cota de peor caso no implica un crecimiento lineal observado: muchas comparaciones pueden fallar pronto. El Suffix Array puede examinar prefijos más largos al aumentar $m$, con cota $O(m \log n)$, sin que aumente la memoria del índice.

#### 4.3.3. Efecto del número de ocurrencias

- **Variable:** $z$, el número de ocurrencias, incluidas las superpuestas. Se fijarán $n = 50$ MB y $m = 4$ bytes en el corpus inglés.
- **Selección:** se contarán las subcadenas distintas de cuatro bytes y se elegirán treinta patrones presentes distribuidos entre frecuencias bajas, intermedias y altas. Se dividirá el rango de frecuencias observadas en tres grupos sobre escala logarítmica y se tomarán diez patrones por grupo. Se añadirán diez patrones ausentes de igual longitud, cuya ausencia se comprobará. Las frecuencias reales, no valores impuestos, serán el eje horizontal; si algún grupo no aporta diez patrones, se reportarán todos los disponibles y su cantidad.
- **Caso extremo:** se usará además $T = A^n$, con los cinco tamaños del primer experimento, buscando $P = A$ y $P = B$. Ambos patrones tienen $m = 1$; se compararán $z = n$ y $z = 0$ por separado del corpus inglés.
- **Resultados:** tiempo por consulta frente a $z$, con $n$ y $m$ fijos en cada comparación. Los casos unarios se mostrarán en otra figura, contrastando ambos patrones para cada tamaño.

**Hipótesis.** Rabin–Karp tenderá a aumentar su tiempo con $z$, pues cada coincidencia real exige verificar los $m$ caracteres. KMP seguirá siendo lineal en $n$, aunque pueden variar sus constantes. La consulta del Suffix Array no enumera coincidencias, por lo que no se espera un costo proporcional a $z$. En fuerza bruta, más coincidencias implican más comparaciones completas, pero los intentos fallidos también influyen. En el caso unario con $m = 1$, fuerza bruta, KMP y Rabin–Karp seguirán siendo lineales en $n$: muchas coincidencias no bastan por sí solas para producir un costo cuadrático.

*Consideración:* en texto natural, patrones con frecuencias distintas también difieren en sus caracteres y prefijos. Este experimento estudia la relación observada entre frecuencia y tiempo, no un efecto causal aislado de $z$. Se usarán varios patrones por grupo para no depender de una única subcadena. La memoria auxiliar no debería variar con $z$, pues solo se devuelve un contador.

## Referencias

[1] Implementación de Suffix Array, repositorio del curso EDAA. https://github.com/jfuentess/edaa/blob/main/strings/suffix_array.cpp.

[2] Implementación de KMP, repositorio del curso EDAA. https://github.com/jfuentess/edaa/blob/main/strings/kmp.cpp.

[3] Implementación de Rabin–Karp, repositorio del curso EDAA. https://github.com/jfuentess/edaa/blob/main/strings/rk.cpp.

[4] Repositorio de textos Pizza & Chili. http://pizzachili.dcc.uchile.cl/texts.html.

[5] Lovera, L. (LELE). (s. f.). *uhr: generic time performance tester* [Código fuente]. Plantilla de medición de la ayudantía, disponible en `utils/uhr.cpp` del repositorio de estos boletines. Referencia del autor utilizada en el Boletín 01: https://github.com/leonardlover.